

CSARCH2 Virtual Exhibit Proposal

Group Name: Project Y2K

Topic Theme: The Y2K Bug in the 20th Century

GitHub Link: <https://github.com/Metthy1871/CSARCH2-Virtual-Exhibit-Proposal/blob/main/README.md>

Group Members

- Adrian Matthew Dee
- Alain Zuriel Marcos
- Elkan La Madrid
- Jenrick Lim
- Kent Lopez

Tech Stack

Frontend

Technology	Role
Astro 6	Primary framework; required by the project template; handles museum page rendering
React (JSX)	All interactive exhibit elements - simulations and educational games as reusable JSX components embedded via MDX
MDX	Main exhibit content page; combines written content with embedded React components
CSS	Styling; used to replicate the visual aesthetic of late-1990s computing environments

1. Astro 6

Astro will serve as the primary framework for our virtual exhibit. It is the required framework for the project template and will be responsible for rendering the museum page.

Will be used for:

- Easy integration of React components
- Compatible with the central museum website architecture

2. React (JSX)

React will be used to develop all the interactive exhibit elements. The project's simulations and educational games will be implemented as reusable JSX components, displayed in the MDX file. In addition, most of us have experience with using React JSX.

Will be used for:

- Year Counter Simulator Component (Interactive Simulator)
- Midnight Crisis Fix-It Game Component (Interactive Game)

- Interactive Timeline Component (Interactive Timeline)

3. MDX

MDX will be used to create the main exhibit content page. It combines the introduction and technical explanations with embedded React components for interactive demonstrations.

Will be used for:

- Easy content and component placement manipulation
- Integration between text and interactive elements

4. CSS

Traditional CSS will be used to replicate the visual style of late-1990s computing environments. In addition, most of us have experience with using traditional CSS.

Planned design elements:

- Create a design with a 1990s technology theme (an early version of Windows) to replicate what the User Interface looked like at the time of the Y2K bug
- CRT monitor design
- Retro terminal interfaces
- Warning screens for interactive simulation
- Responsive layouts for desktop and mobile devices
- Animations throughout the webpage

Backend

Technology	Role
Node.js	Runtime environment for local development and Astro deployment
Express	Included as a contingency for additional backend functionality if needed

1. Node.js

Node.js will provide the runtime environment for the backend of our local development and for the deployment of the Astro application. Moreover, it is also very compatible and is required by Astro.

Will be used for:

- Compatibility with Astro, since it's required by Astro
- Supports modern JavaScript tooling and is compatible with React (JSX)

2. Express

Express will be used only if additional backend functionality becomes necessary. In addition, the majority of the team has experience in utilizing Express with Node.js.

Potential uses:

- Recording high scores for the interactive games
 - Tracking visitor statistics
-

I. Proposed Structure

1. Introduction (Story Hook)

As the final seconds of 1999 ticked away, the world stood on the edge of a digital cliff, fearing that the technology powering modern civilization would fail the moment 2000 began. This fear became known as the Y2K crisis or the "Millennium Bug". Y2K was caused by storing years with only two digits (storing 1999 as "99"). Moreover, many feared that computers interpreted "00" as 1900 instead of 2000. This simple problem threatened to crash the systems that controlled everything, such as bank accounts and electricity, which resulted in people losing money or power. Eventually, the Y2K crisis became a crucial lesson in the importance of preparing for technological risks before they develop into a global threat.

2. Technical Explanation (CSARCH Core)

a. 2-Digit Year Storage

In the early years of computing, storage was extremely expensive. Programmers stored years as only 2 digits to conserve space. COBOL, the dominant language of the era, reflected this in its data definitions via 6-digit date fields: `PIC 9(6)`.

b. Memory Limitations

The problem existed at the hardware level as well — not just in code. BIOS chips and Real-Time Clock (RTC) chips stored dates in a 2-digit BCD (Binary Coded Decimal) format.

c. Ambiguity

Since "00" was ambiguous, machines encountered problems because there was no proper definition for the year 2000.

```
1999 → "99"  
2000 → "00" ← interpreted as 1900
```

This caused three main failure modes:

1. **Negative arithmetic** — e.g., a person born in 1950 would be calculated as -50 years old
2. **Wrong sort order** — data from 2000 onwards would appear *before* 1999 data ("99" > "00")
3. **Crashes** — some systems rejected "00" as invalid input entirely

3. Timeline (Visual Section)

Period	Event
1970–1990	Systems designed with 2-digit year shortcuts
1995–1999	Public and industry panic begins
1999–2000	Global remediation effort underway

II. Interactive Components

1. Simulation — "The Year Counter: Stop at 2000"

Concept: A system clock simulation counting years from 1990 → 2000.

Gameplay:

- The counter ticks forward: 1990, 1991, 1992 ... 1998, 1999, 2000
- The user can press a button at any time to pause it
- If the counter reaches 2000 → crash animation plays, then resumes after any button press
- After the crash, it loops back to 1990 and repeats

What it teaches:

- The rollover bug threat
 - Integer overflow and misinterpretation
 - System failure as a consequence of data representation
-

2. Game — "Midnight Crisis Fix-it"

Concept: You are a software engineer on December 31, 1999. Fix the systems before midnight.

Gameplay:

- Choose from a list of systems to patch, each costing "time":
 - [1] Banking System
 - [2] Airline Reservation System
 - [3] Hospital Records
 - [4] Power Grid
- A 12-minute real-time timer runs, displayed as 24 hours (2× speed): 23:58 → 23:59 → 00:00

Outcome states:

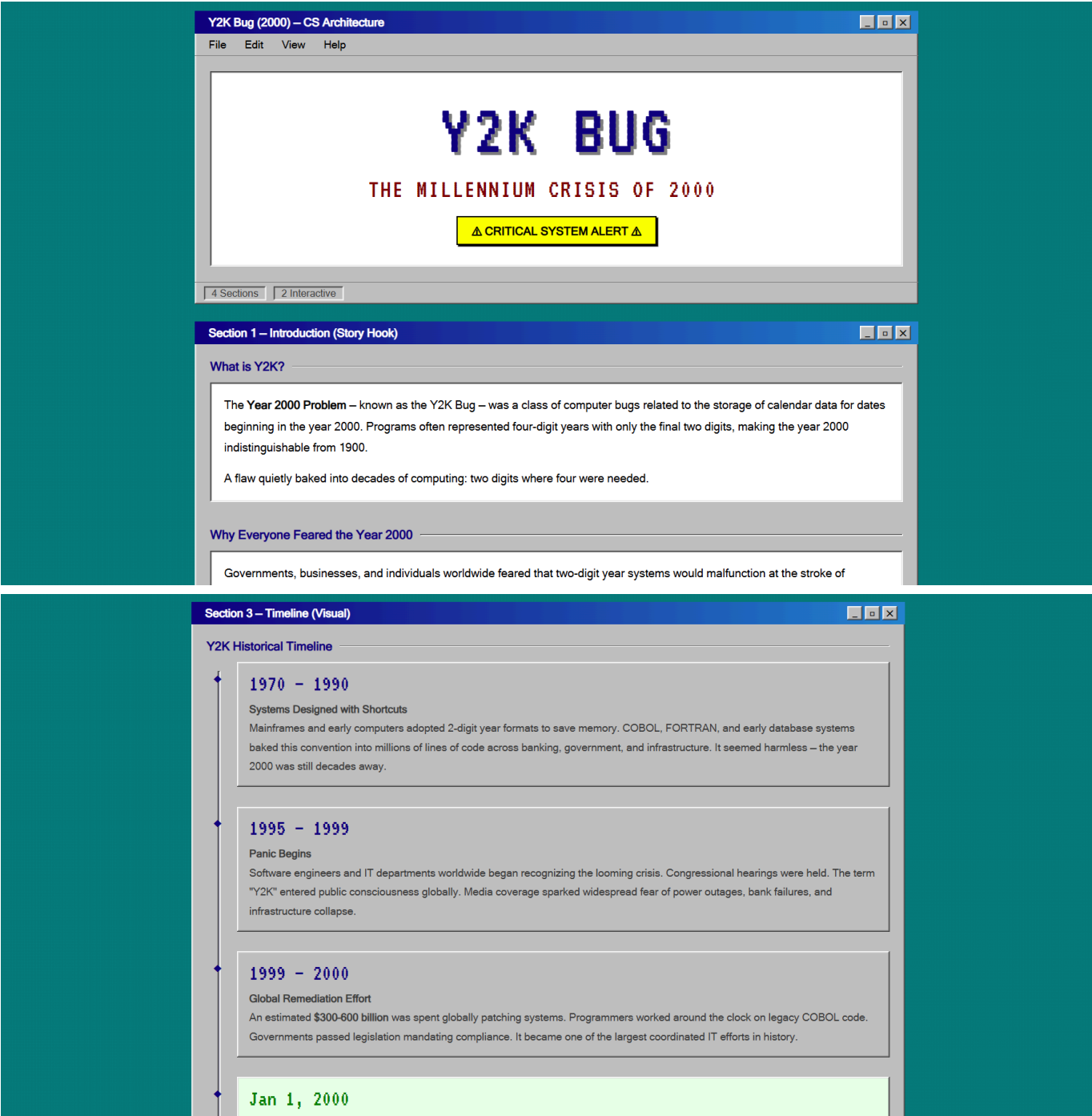
- All systems fixed → **World is Stable** ending
- Systems missed → **System Collapse Simulation**

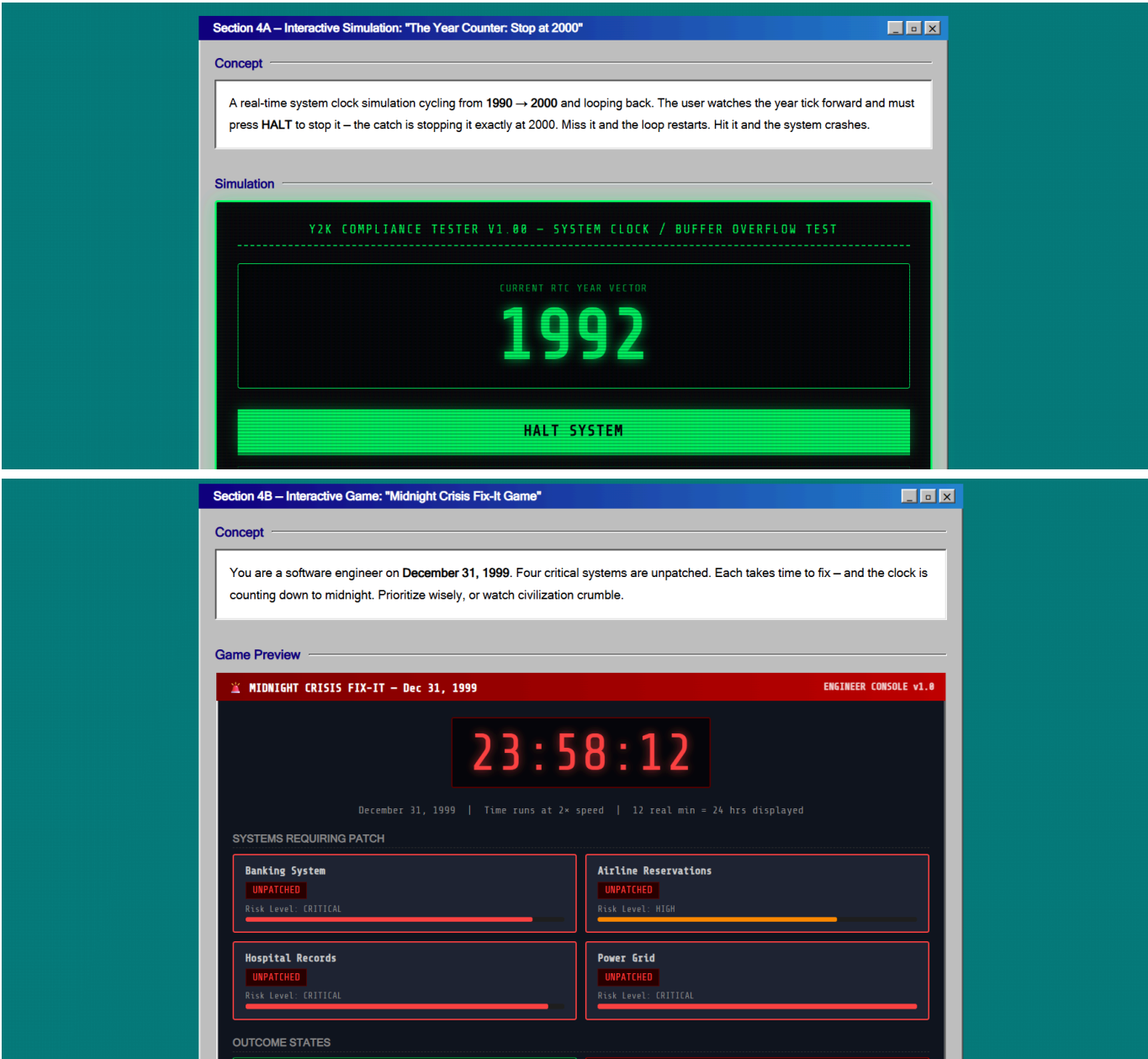
What it teaches:

- Prioritization in system design
 - Real-world engineering constraints
 - Why the Y2K remediation effort cost billions
-

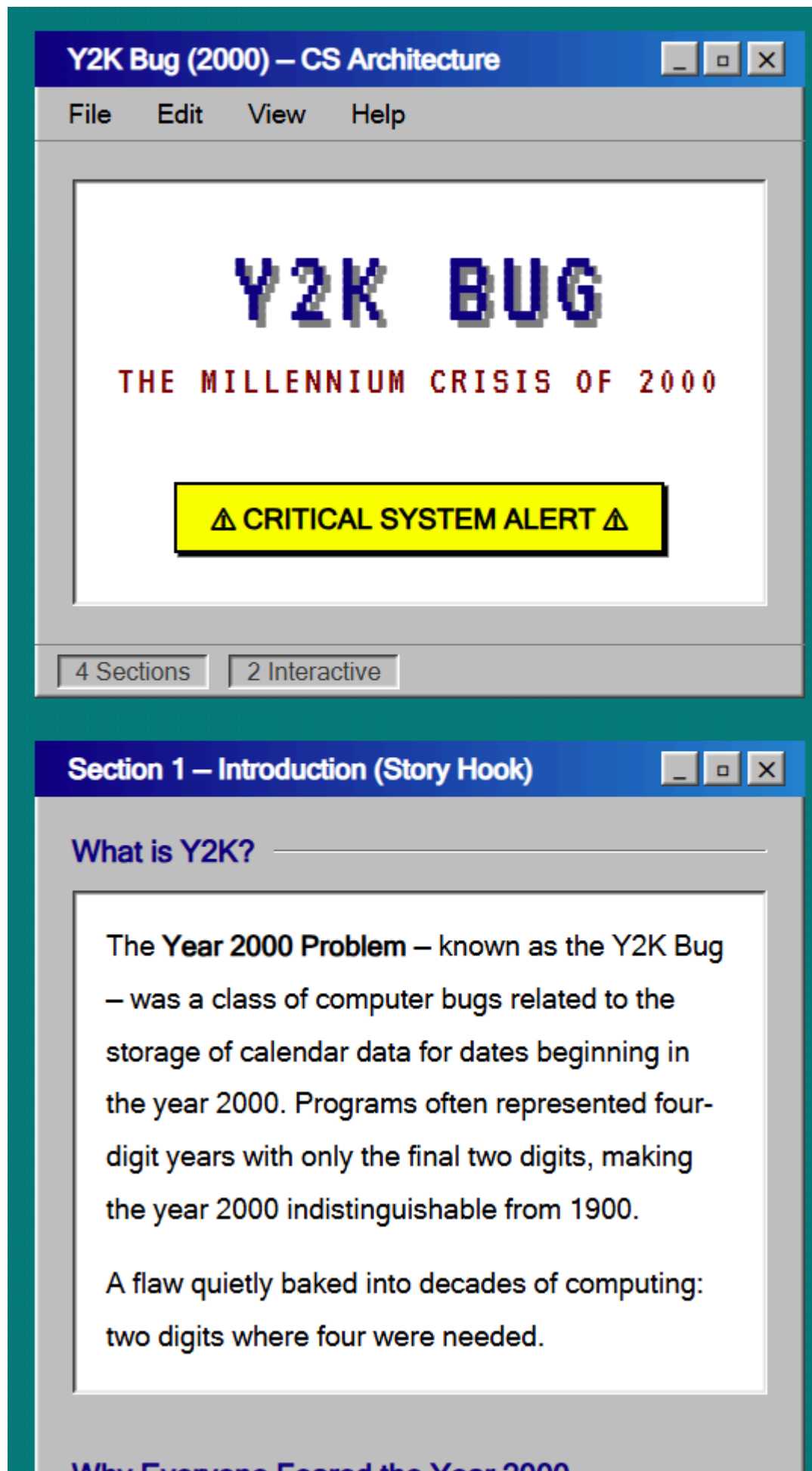
III. Proposed Design Layout

PC Display





Mobile Display



Section 3 – Timeline (Visual)

Y2K Historical Timeline

1970 – 1990

Systems Designed with Shortcuts

Mainframes and early computers adopted 2-digit year formats to save memory. COBOL, FORTRAN, and early database systems baked this convention into millions of lines of code across banking, government, and infrastructure. It seemed harmless – the year 2000 was still decades away.

1995 – 1999

Panic Begins

Software engineers and IT departments worldwide began recognizing the looming crisis. Congressional hearings were held. The term "Y2K" entered public consciousness globally. Media coverage sparked widespread fear of power outages, bank failures, and infrastructure collapse.

1999 – 2000

Global Remediation Effort

An estimated \$300-600 billion was spent

Section 4A – Interactive Simulation: "The Year Counter: Stop at 2000"

Concept

A real-time system clock simulation cycling from 1990 → 2000 and looping back. The user watches the year tick forward and must press **HALT** to stop it – the catch is stopping it exactly at 2000. Miss it and the loop restarts. Hit it and the system crashes.

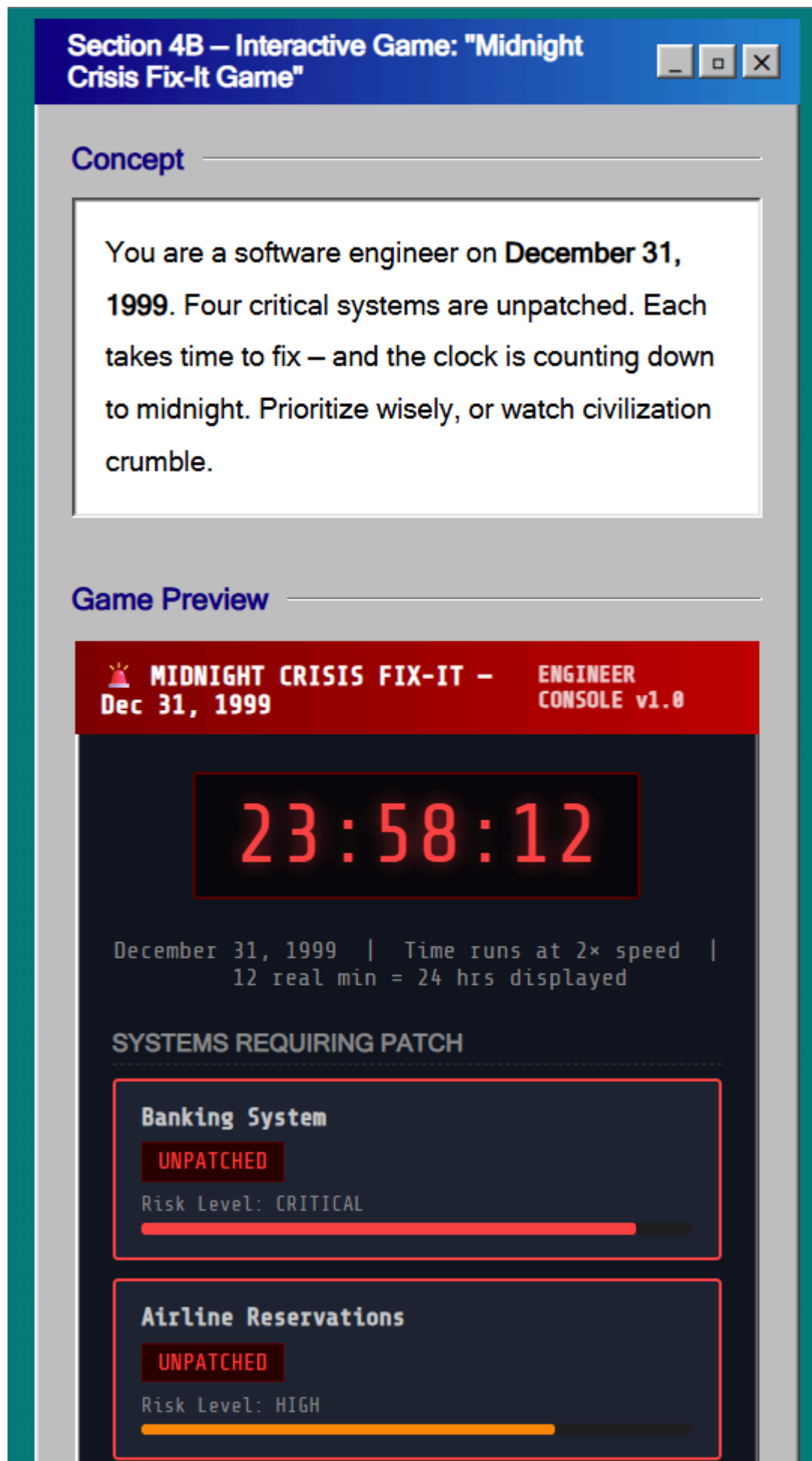
Simulation

Y2K COMPLIANCE TESTER V1.00 -
SYSTEM CLOCK / BUFFER OVERFLOW
TEST

CURRENT RTC YEAR VECTOR

1994

HALT SYSTEM



IV. Planned JSX Components

Component	Purpose
<code>Y2KTimeline.jsx</code>	Displays the visual timeline of Y2K events from 1970 through the global remediation effort
<code>YearCounter.jsx</code>	Simulates the year rollover from 1990–2000 with crash animation and loop logic
<code>CrisisManager.jsx</code>	Manages the Midnight Crisis Fix-It game, including the system selection, time cost, timer, and outcome states